

Buổi 1

Nền tảng Python, bản Python

Kiến thức, mười bài tập và mười code mẫu

Ngày 23 tháng 9 năm 2026

TS. Đỗ Phúc Hảo

Vì sao thư mục vẫn tên là **01-nen-tang-cpp**. Đây là lý do kỹ thuật chứ không phải sơ suất. Bộ chấm của nhánh này mượn bộ test từ nhánh C++ theo đúng tên chủ đề: nó tìm `../topics/01-nen-tang-cpp/tests/bai0X/`. Đổi tên thư mục là mất sạch 41 test và mười đề bài mất chỗ dựa. Nên thư mục giữ tên cũ, còn nội dung thì không: file bài giảng tên `NEN-TANG-PYTHON-PY.md`, và `cau-hinh.json` khai tiêu đề là "Nền tảng Python, bản Python".

Và nội dung thật sự khác, không phải dịch. Bản C++ dành cả Phần I cho `ios::sync_with_stdio`, bảng ngưỡng tràn của `int` và `long long`, luật comparator chặt, và cờ `-fsanitize`. Trong Python **không một mục nào trong số đó tồn tại**. Nếu tôi dịch chúng sang tiếng Việt và thay vector bằng list thì bạn sẽ học cách đề phòng những thứ không có thật, và tệ hơn, chúng chiếm mất chỗ của cạm bẫy thật.

Mười đề bài và bộ test thì giữ nguyên **không đổi một chữ**. Đó là cố ý: cùng một bài toán, soi từ hai ngôn ngữ, là cách rõ nhất để thấy chỗ nào là khó của bài toán và chỗ nào chỉ là thuế của ngôn ngữ.

Phần	Nội dung
I	Kiến thức, mười một mục
II	Mười đề bài A tới J
III	Gợi ý phân tầng và lời giải từng bài
IV	Mười code mẫu đầy đủ

Về mọi con số trong tài liệu này. Tất cả đều do tôi đo trên **máy dựng tài liệu này**: Python 3.9.6, macOS arm64. Cách đo: chạy tiến trình con, bấm giờ bằng `time.perf_counter()` trong cùng một tiến trình, chạy ít nhất ba lượt **xen kẽ** hai cách rồi lấy lượt **nhANH NHẤT**, và trừ đi thời gian khởi động trình thông dịch đo riêng, khoảng 0,011 giây. Đây là số của một máy cụ thể, không phải hằng số của ngôn ngữ. Tỷ lệ giữa hai cách thì bền hơn con số tuyệt đối, nên hãy đọc tỉ lệ.

Phần I. Kiến thức

1. Mô hình số, phần Python cho không và phần Python thu lại

Đây là mục quan trọng nhất của cả buổi, vì nó quyết định năm trong mười bài.

Số nguyên là vô hạn. Không có `int`, không có `long long`, không có `__int128`, không có ngưỡng. 10^{18} , 10^{100} , tích của hai số 10^9 đều đúng tuyệt đối. Cả Phần I mục 2 của bản C++, tức bằng ngưỡng an toàn và ba tình huống tràn kinh điển, **biến mất sạch**. Bài A, bài C, bài E và bài H của buổi này đều được C++ ra đề để phạt tràn số; ở Python chúng mất hẳn một nửa lý do tồn tại.

Số thực thì vẫn là double. float của Python là IEEE 754 64 bit: `sys.float_info.mant_dig` bằng **53**, `sys.float_info.dig` bằng **15**. Nghĩa là từ $2^{53} = 9007199254740992$, tức khoảng $9 \cdot 10^{15}$, số nguyên liên nhau không còn phân biệt được nữa.

Đo được, không phải suy đoán:

Biểu thức	Kết quả
<code>float(10**18) == 10**18</code>	True, vì 10^{18} tình cờ biểu diễn được đúng
<code>int((10**18 + 1) / 1)</code>	1000000000000000000, lệch 1 đơn vị
<code>int(float(999999937 * 999999937))</code>	lệch 1 so với tích đúng 999999874000003969
<code>int(float(2**53 + 1))</code>	9007199254740992, lệch 1

Dòng đầu là cái bẫy trong cái bẫy: thử đúng một giá trị rồi kết luận "float chịu được 10^{18} " là sai, vì 10^{18} là một trong số ít giá trị ở tầm đó biểu diễn được chính xác. Hàng xóm của nó thì không.

/ luôn ra float. Kể cả khi hai vế đều nguyên và chia hết. $7/2$ cho 3.5 ; $10^{18}/1$ cho $1e+18$ chứ không phải một số nguyên. Chỗ bản C++ cảnh báo "tổng vượt long long", bản Python phải cảnh báo "lỡ tay viết / thay //", vì mất chính xác bắt đầu từ $9 \cdot 10^{15}$, tức **ngay trong tầm dữ liệu của bài A**.

// và % làm tròn về âm vô cùng. Đây là khác biệt hành vi lớn nhất giữa hai ngôn ngữ, và nó cắt cả hai chiều:

Biểu thức	Python	C++
<code>-7 // 2</code>	-4	-3
<code>-7 % 2</code>	1	-1
<code>7 // -2</code>	-4	-3
<code>7 % -2</code>	-1	1
<code>-1 % 5</code>	4	-1

Luật một câu: **phần dư của Python mang dấu của số CHIA**, phần dư của C++ mang dấu của số **bị chia**.

Hệ quả tốt: bài H của buổi này, vốn là bài tập chuẩn hoá $((x \% n) + n) \% n$ của C++, ở Python chỉ còn $x \% n$. Công thức chuẩn hoá trở thành mã chết.

Hệ quả xấu, và đây mới là chỗ mất điểm: khi bạn muốn **cắt về không** như C++, // làm sai. `-7 // 2` cho -4, tức làm tròn **ra xa** số không. Bài "có bao nhiêu nhóm đủ chỗ" với số âm mà dùng // là sai một đơn vị. Cách chữa đúng là `-(-a // b)`, không phải `int(a / b)`: cái sau đi qua float và mất chính xác từ $9 \cdot 10^{15}$.

Căn bậc hai nguyên. `int(math.sqrt(n))` đi qua float. Dùng `math.isqrt(n)`, chính xác tuyệt đối ở mọi cỡ.

2. Vào ra, ba cách và số đo của chúng

Đọc $2 \cdot 10^5$ số nguyên rồi cộng lại, cùng một file, đo xen kẽ, lượt nhanh nhất, đã trừ khởi động:

Cách đọc	Thời gian	So với cách nhanh nhất
<code>sys.stdin.buffer.read().split()</code>	0,013 s	1,0
<code>sys.stdin.readline()</code> trong vòng lặp	0,020 s	1,6
<code>input()</code> trong vòng lặp	0,130 s	10,0

`input()` chậm hơn đúng một bậc. Với $2 \cdot 10^5$ dòng, một mình phép đọc đã tốn 0,13 giây trước khi bạn tính gì cả; với 10^6 dòng nó là nửa giây.

Phía ra, in $2 \cdot 10^5$ dòng:

Cách ghi	Thời gian
gom list rồi <code>"\n".join</code> và ghi một lần	0,016 s
<code>print()</code> từng dòng	0,024 s

Chênh lệch phía ra nhỏ hơn phía vào nhiều, chỉ 1,5 lần chứ không phải 10 lần. Nói `print()` trong vòng lặp là nguyên nhân TLE thì quá lời; nó là một khoản thuế nhỏ, và lý do thật để gom vào list là code sạch hơn.

Khi nào không được dùng `read().split()`. Khi bài về **dòng** chứ không về **token**, như bài I của buổi này. Lúc đó cắt theo `"\n"` và **chỉ** theo `"\n"`.

Một điều bản C++ phải dạy mà Python không cần: tách từ trong một dòng. C++ cần `istringstream` để bỏ qua khoảng trắng thừa; `split()` không đổi số của Python đã gộp mọi cụm khoảng trắng và bỏ mảnh rỗng hai đầu, nên `" a bb "` cho đúng `["a", "bb"]`. Nhưng `split(b" ")` có đổi số thì không gộp gì cả và trả về cả mảnh rỗng. Một dấu cách trong ngoặc là khác biệt giữa AC và WA.

3. Tra biến cục bộ nhanh hơn tra biến toàn cục

Cùng một vòng lặp $5 \cdot 10^6$ lượt, một bản đặt trong hàm `main()` và một bản viết ở mức module:

Chỗ đặt	Thời gian
trong hàm, biến cục bộ	0,096 s
ở mức module, biến toàn cục	0,186 s

Gần gấp đôi, và không đổi một dòng logic nào. Lý do: biến cục bộ trong một hàm được trình biên dịch bytecode gán **số thứ tự** và tra bằng chỉ số mảng; biến toàn cục tra bằng **từ điển** của module. Đó là toàn bộ lý do khung chuẩn của kho này bắt mọi thứ nằm trong `main()`.

Cùng một cơ chế cho một mẹo dùng suốt: **buộc hàm thư viện vào tên cục bộ trước vòng lặp**. Viết `tim = bisect_right` rồi gọi `tim(...)` thay vì `bisect.bisect_right(...)` bỏ được một lần tra từ điển module mỗi lượt. Bài D dùng đúng mẹo này.

4. Chi phí thật của list, dict, set, tuple

Dựng $2 \cdot 10^5$ phần tử, một phần tử một lượt:

Cách dựng	Thời gian
<code>list.append(i)</code>	0,0051 s
<code>set.add(i)</code>	0,0059 s
<code>dict[i] = i</code>	0,0064 s
<code>list.append((i, i))</code> dựng tuple mỗi lượt	0,0116 s

Tra cứu 10^6 lần:

Cách tra	Thời gian
<code>a[i]</code> trên list	0,036 s
<code>d[k]</code> trên dict	0,044 s

Kết luận đáng nhớ: **bốn cấu trúc này gần như cùng giá khi dùng đúng chỗ**. Hơn kém nhau vài chục phần trăm, không phải vài bậc. Chuyện vài bậc chỉ xảy ra khi bạn dùng **sai** cấu trúc, và lúc đó chênh lệch không còn là vài chục phần trăm nữa:

Việc cần làm	Cách đúng	Cách sai	Số đo
hỏi <code>x</code> có trong tập không, 10^5 lần trên 10^4 phần tử	set	list	0,0016 s so với 3,85 s
lấy ra từ đầu hàng, 10^5 lần	<code>deque.popleft()</code>	<code>list.pop(0)</code>	0,005 s so với 0,559 s
giữ một dãy luôn sắp, 10^5 lần chèn	sắp một lần ở cuối	<code>bisect.insort</code>	0,038 s so với 0,625 s

Dòng thứ nhất là khoảng **2470 lần**, và con số ấy có một lai lịch đáng kể. Lướt đo đầu tiên của tôi cho 3867 lần, vì phía set chỉ mất 0,001 giây, tức chạm sàn phân giải của phép đo. Tôi cho phía set làm gấp mười lần công việc rồi chia lại, và con số thật là 2470. Bài học rộng hơn con số: khi một tỉ lệ đo được nằm ở hàng nghìn, hãy nghi bộ đo của mình trước khi tin vào phát hiện. Dù vậy 2470 lần vẫn là khác biệt duy nhất trong bảng này đủ lớn để một mình gây TLE.

Ba dòng sau lại cho thấy một điều ngược với lời khuyên thường nghe: `list.pop(0)` **chậm 112 lần** so với deque nhưng vẫn chỉ hết 0,559 giây, tức vẫn kịp trong giới hạn 3 giây của buổi này. Lý do là phép dời mảng của CPython là **một lệnh memmove** ở mức C trên một mảng đủ nhỏ để nằm trong cache. Nói "dùng list làm hàng đợi thì TLE" là dạy một con số không chịu nổi việc bị kiểm lại. Vẫn dùng deque, nhưng vì nó là cấu trúc đúng, không phải vì một lời dọa.

Dòng cuối là lý do bài J của buổi này chạy 0,707 giây chứ không phải 0,04 giây.

5. Bốn thứ chậm bất ngờ, và một thứ hoá ra không chậm

Cấp phát trong vòng nóng. $2 \cdot 10^6$ lượt:

Trong thân vòng lặp	Thời gian
chỉ số học, không tạo đối tượng	0,069 s
tạo một list hai phần tử mỗi lượt	0,114 s
tạo một dict hai khoá mỗi lượt	0,135 s

Tạo một đối tượng nhỏ mỗi lượt cộng thêm khoảng hai phần ba thời gian. Trong một vòng lặp chạy 10^7 lượt thì đó là ranh giới giữa kịp và không kịp.

Cắt list là chép, không phải nhìn. 10^4 phép cắt trên một list 10^4 phần tử:

Phép cắt	Thời gian
<code>a[i:i+2]</code>	dưới 0,001 s
<code>a[i:]</code>	0,042 s

Cùng số phép cắt, khác khối lượng chép: bản thứ hai chép $5 \cdot 10^7$ con trở. Trong C++ bạn quen truyền `begin()` và `end()`; ở Python `a[i:]` **dựng một list mới**. Muốn đi qua đuôi mà không chép thì dùng chỉ số, hoặc `itertools.islice`.

Gọi hàm. $3 \cdot 10^6$ lượt:

Cách viết	Thời gian
mã thẳng trong vòng lặp	0,110 s
gọi một hàm con làm đúng việc ấy	0,163 s

Đặt thêm gần một nửa. Đây là lý do mục ”gỡ hàm con thành mã thẳng” nằm trong danh sách việc phải thử khi một bài chạm giới hạn.

Và thứ hoá ra không chậm: nối chuỗi trong vòng lặp. Lời khuyên kinh điển là $s = s + x$ trong vòng lặp là $O(n^2)$. Tồi đo, và nó **sai** ở dạng thường gặp nhất. Nối $8 \cdot 10^5$ mảnh, mỗi mảnh 20 ký tự:

Cách viết	Thời gian
gom list rồi <code>"".join(list)</code>	0,020 s
<code>s = s + x</code>	0,026 s
<code>s = s + x</code> , nhưng có một tham chiếu thứ hai tới <code>s</code>	122,5 s

Hai dòng đầu gần như bằng nhau, vì CPython có một tối ưu: khi chuỗi `s` chỉ còn **đúng một** tham chiếu, phép cộng cấp phát lại tại chỗ thay vì chép sang chỗ mới. Dòng thứ ba là cùng một vòng lặp, chỉ thêm một dòng `gi = s` giữ lại tham chiếu thứ hai. Tối ưu tắt, và độ phức tạp thật lộ ra. Đo ở ba cỡ để chắc đó là $O(n^2)$ chứ không phải chậm đều:

Số mảnh	Thời gian bản có tham chiếu thứ hai
$5 \cdot 10^4$	0,62 s
$2 \cdot 10^5$	7,56 s
$8 \cdot 10^5$	122,5 s

Gấp bốn số mảnh thì thời gian gấp 12 rồi gấp 16 lần. Đúng hình dạng bình phương.

Bài học không phải ”cứ nổi thoải mái”. Bài học là: tối ưu ấy **không có trong đặc tả ngôn ngữ**, nó phụ thuộc vào một điều kiện bạn không nhìn thấy trong code của mình, và nó tắt lặng lẽ. `"".join(list)` không phụ thuộc vào gì cả. Dùng `join`, nhưng hãy biết mình đang tránh cái gì.

6. Đệ quy: hai giới hạn, và cái thứ hai mới giết bạn

Giới hạn mềm là `sys.getrecursionlimit()`, mặc định **1000**. Vượt là `RecursionError`, có truy vết, biết ngay dòng nào. Một DFS trên đồ thị 10^5 đỉnh chạm nó ngay.

Phản xạ đầu tiên của mọi người là `sys.setrecursionlimit(10**9)`. Việc đó gỡ giới hạn mềm và để lộ **giới hạn cứng**: ngăn xếp thật của hệ điều hành. Tồi đo ngưỡng chết trên máy này, bằng cách chia đôi khoảng sống và khoảng chết:

Hình dạng khung	Sâu nhất còn sống	Nông nhất đã chết
hàm tối giản, một tham số	32.710 khung	32.903 khung
hàm DFS thật, ba tham số, có vòng lặp và bốn biến cục bộ	32.694 khung	32.753 khung

Hai hàng gần như trùng nhau, nên trên máy này ngưỡng là **khoảng 32.700 khung** và gần như không phụ thuộc vào việc khung nặng hay nhẹ.

Cái đáng sợ là **cách nó chết**: mã thoát -11, tức `SIGSEGV`, mà vỏ lệnh báo thành **139**. Không có `RecursionError`. Không có truy vết. **Không in ra một chữ nào**. Trên máy chấm nó hiện thành RE hoặc thành một kết quả rỗng, và bạn không có manh mối nào để lần.

Ba điều rút ra:

- Con số 32.700 là **của máy này**, phụ thuộc kích thước ngăn xếp của hệ điều hành. Đừng mang nó sang máy khác như một hằng số. Một agent khác đo trên cùng máy này bằng một phép thử khác ra khoảng 21.800; chênh lệch ấy chính là lý do không được coi nó là hằng số. Cái **bền** là hình dạng của hiện tượng: có một ngưỡng, nó nằm ở hàng chục nghìn chứ không hàng triệu, và vượt nó thì chương trình chết cầm.
- Nâng giới hạn mềm lên vài chục nghìn thì được; nâng lên 10^{**9} là đổi một lỗi **có truy vết** lấy một lỗi **không truy vết**. Nếu buộc phải nâng, hãy nâng vừa đủ, ví dụ `sys.setrecursionlimit(300000)`, và biết rằng con số ấy chưa chắc chạm tới được.
- Cách chữa thật không phải là nâng giới hạn mà là **khử đệ quy**: viết DFS bằng một list làm ngăn xếp tường minh. Cách đó không có ngưỡng nào cả, vì ngăn xếp nằm trên heap.

Bản C++ có một dòng tương ứng trong danh sách cạm bẫy, mục 5 của nó: "đệ quy sâu 10^6 mức làm tràn stack". Hình dạng giống nhau, nhưng con số khác hẳn: 10^6 khung C++ và $3 \cdot 10^4$ khung Python. Một khung Python đắt hơn một khung C++ khoảng ba mươi lần về ngăn xếp.

7. Bản sao nông, bản sao sâu, và `[[0] * m] * n`

Đây là nơi Python im lặng nhất.

```
a = [[0] * 3] * 2
a[0][0] = 5
# a is now [[5, 0, 0], [5, 0, 0]]
```

`[x] * n` **lặp lại tham chiếu**, không nhân bản đối tượng. Với `[0] * m` thì vô hại vì số nguyên bất biến. Với `[[0] * m] * n` thì `n` hàng là **cùng một list**, và gán vào hàng 0 đổi mọi hàng. Không cảnh báo, không lỗi, chỉ có kết quả khó hiểu.

Dạng đúng là một list comprehension, vì nó chạy biểu thức lại `n` lần:

```
a = [[0] * m for _ in range(n)]
a[0][0] = 5
# a is now [[5, 0, 0], [0, 0, 0]]
```

Cùng một cơ chế, một mặt khác: `g[:]`, `list(g)` và `g.copy()` đều là **bản sao nông**. Chúng chép lớp ngoài, còn các list con vẫn là cùng đối tượng:

```
g = [[1, 2], [3, 4]]
n1 = g[:]
n1[0][0] = 99
# g is now [[99, 2], [3, 4]]
```

Muốn tách thật thì `copy.deepcopy(g)`, hoặc dựng lại bằng comprehension `[hang[:] for hang in g]`. `deepcopy` đi theo mọi tham chiếu nên nó đắt; trong thi đấu, comprehension gần như luôn đủ.

Quy tắc một câu: **mỗi lần bạn thấy `*` nhân một list, hãy hỏi phần tử của nó có bị sửa tại chỗ không.**

8. Sắp xếp: Python nhận KEY, C++ nhận COMPARATOR

Bản C++ dành nguyên một mục cho luật comparator chặt, và cảnh báo rằng vi phạm nó làm sort **đọc ra ngoài mảng và sập**, mà chỉ sập với mảng đủ lớn. Trong Python **không có comparator**, nên không có luật ấy để vi phạm và không có cách nào làm sort sập. Đây là món quà lớn thứ hai sau số nguyên vô hạn.

Đổi lại, bạn phải học cách nghĩ bằng khoá:

- **Xếp tầng**: dùng tuple. `key=lambda x: (a, b, c)` so sánh từ điển, đúng như comparator nhiều tầng, và không có cách nào viết sai thứ tự tầng.
- **Tầng giảm dần xen với tầng tăng dần**: với số thì đảo dấu, `key=lambda x: (-diem, ten)`. Với xâu thì không đảo dấu được, nên sắp **hai lần**: sắp theo tầng tăng trước, rồi sắp lại theo tầng giảm. sort của Python **ổn định**, nên lần sắp sau giữ nguyên thứ tự lần trước. Bài G dùng cách thứ nhất.

- **Khi thứ tự không viết được thành khoá:** có `functools.cmp_to_key`, nhưng nó trả giá. Nó bọc mỗi phần tử vào một đối tượng và chạy một lệnh gọi Python cho **mỗi phép so sánh**, tức khoảng $n \log n$ lần, thay vì n lần cho khoá. Đo trên $2 \cdot 10^5$ phân số của bài E:

Cách sắp	Thời gian
khoá số nguyên tự dựng	0,210 s
<code>functools.cmp_to_key</code> với comparator nhân chéo	0,443 s
<code>fractions.Fraction</code> làm khoá	1,690 s

Cả ba đều **chính xác**. Chỉ cái thứ nhất là hình dạng Python được xây để chạy. Bài E của buổi này chính là bài tập biến một thứ tự "chỉ so sánh được" thành một khoá.

9. Ước lượng độ phức tạp, bảng của Python

Bản C++ có bảng "n tối đa và độ phức tạp cho phép". Bảng ấy **không dùng lại được** vì mốc của nó là 10^8 phép tính mỗi giây, còn Python thuần đi qua trình thông dịch từng lệnh.

Đo trên chính các phép đo ở trên: một vòng lặp Python thuần làm được khoảng $5 \cdot 10^7$ lượt cộng đơn giản mỗi giây trên máy này. Nhưng một vòng lặp chạy ở **mức C**, tức `sum`, `max`, `sorted`, `Counter`, `bisect`, `"".join`, nhanh hơn khoảng một bậc.

Số phép ước lượng	C++	Python thuần	Python đẩy được xuống mức C
tới 10^6	thoải mái	thoải mái	thoải mái
10^7	thoải mái	vừa đủ	thoải mái
10^8	vừa đủ	không kịp	vừa đủ
từ 10^9	không kịp	không kịp	không kịp

Cột thứ ba là chỗ quyết định. Câu hỏi đúng khi đọc giới hạn của một đề Python không phải "thuật toán này bao nhiêu phép" mà là **"bao nhiêu trong số đó chạy trong vòng lặp Python của tôi, và bao nhiêu nằm trong một lời gọi hàm dựng sẵn"**. Cùng $O(n \log n)$, một bản `sorted(a)` và một bản tự cài merge sort cách nhau một bậc.

10. Mười bài này phạt gì ở C++, và phạt gì ở Python

Đây là bảng đáng giá nhất của tài liệu. Cột giữa là lý do bản C++ ra mười đề này; cột phải là thứ thật sự chờ bạn.

Bài	Bẫy của bản C++	Bẫy của bản Python
A	trần int ở tổng và ở tích, long long $p = a*b$ vẫn sai	biến mất. Còn lại: tích lớn nhất có thể là hai số âm; và cho số đi qua float thì mất chính xác từ $9 \cdot 10^{15}$
B	quên erase sau unique, hoặc unique khi chưa sort	biến mất. sorted(set(a)) là cả bốn bước. Thay vào: chọn dict hay bisect là chọn $O(n)$ hay $O(n \log n)$
C	đáp án $2 \cdot 10^{10}$ tràn int; unordered_map bị hack	biến mất cả hai. Thay vào: d[k] ném KeyError, mà defaultdict[k] lại tạo khoá
D	nhầm lower_bound với upper_bound	y hệt, đổi tên thành bisect_left với bisect_right
E	double gộp hai phân số khác nhau; comparator không chặt gây sập	mất chính xác y hệt; comparator không chặt biến mất vì Python không có comparator. Thay vào: cmp_to_key chậm gấp đôi khoá
F	priority_queue mặc định là max-heap, phải viết đủ ba tham số	biến mất, heapq chỉ có min-heap. Thay vào: khi cần max-heap thì phải tự đảo dấu
G	comparator hai tầng viết sai hình dạng	biến mất. Thay vào: tầng giảm dần phải đảo dấu, và xâu thì không đảo dấu được
H	dấu của % với số âm, và thời điểm lấy dư kéo tràn	biến mất cả hai. Đây là bài duy nhất Python dễ hơn hẳn. Nhưng // làm tròn về âm vô cùng là cái bẫy ở bài khác
I	quên cin.ignore() nên dòng đầu đọc ra rỗng	hình dạng giống, nguyên nhân khác: read().split() xoá mất cấu trúc dòng, và split(b" ") có đối số thì đếm cả mảnh rỗng
J	lower_bound tự do chạy tuyến tính; hai ca biên iterator; tập rỗng	Python không có tập hợp có thứ tự. list cộng bisect là thứ thay thế, và nó trả giá bằng memmove mỗi lần chèn

Đếm lại cho rõ, vì con số mới là thứ dùng được:

Số phạm ở Python	Bao nhiêu bài	Là bài nào
biến mất hoàn toàn	6	A, B, C, F, G, H
sống nguyên vẹn	2	D, và nửa double của E
đổi hình dạng, vẫn còn đó	2	I, J

Sáu bài ở hàng đầu mất bẫy vì đúng ba nguyên nhân, và cả ba đều là đặc điểm của ngôn ngữ chứ không phải may mắn: số nguyên vô hạn, không có comparator nên không có hợp đồng để vi phạm, và phép dư mang dấu của số chia. Hai bài ở hàng giữa là lỗi **đọc đề và hiểu số**, không phải lỗi ngôn ngữ, nên đổi ngôn ngữ không cứu được.

Nhưng danh sách cạm bẫy không ngắn đi. Chỗ nào bản C++ nhả ra thì bản Python thu lại một thứ khác, và ba cái đắt nhất trong bảng trên là: list dùng như set chậm khoảng 2470 lần, đệ quy chết cam ở khoảng 32.700 khung, và $[[0] * m] * n$ chia sẻ hàng mà không báo gì.

11. Bộ khung tự hỏi khi gặp đề Python mới

- Dữ liệu là **token** hay là **dòng**? Có khoảng trắng nào mang nghĩa không?
- Có phép chia nào không? Cần `//` hay `/`? Có số **âm** đi qua `//` hoặc `%` không?
- Có số nguyên nào đi qua `float`, `math.sqrt`, hay `sum` của list số thực không? Nó có thể vượt $9 \cdot 10^{15}$ không?
- Trong vòng lặp nóng nhất: có tạo đối tượng mới mỗi lượt không? Có cắt list không? Có gọi hàm con không? Có tra tên toàn cục không?
- Cấu trúc nào đang được dùng để hỏi "có chứa không"? Nếu là `list` thì đổi sang `set` ngay.
- Có đệ quy không? Độ sâu tối đa theo giới hạn của đề là bao nhiêu? Trên 10.000 thì viết bản lặp.
- Có list lồng list không? Nó được dựng bằng `comprehension` hay bằng `*`?
- Thứ tự sắp xếp viết được thành **khóa** không? Nếu không thì vì sao không?

Phần II. Mười bài tập

Mười bài theo dạng đề ICPC, tăng dần. Mỗi bài gài đúng một cạm bẫy đã nói trong Phần I. Cứ code trước, đừng mở gợi ý.

Giới hạn thời gian **3 giây** cho mọi bài, bộ nhớ 256 MB, vào ra chuẩn. Con số 3 giây là **đo ra**, không phải nhân giới hạn 1 giây của bản C++ với một hằng số: bài chậm nhất của bộ mẫu là bài J, đo được 665 tới 767 ms qua nhiều lượt tùy tải máy, tức dưới một phần tư giới hạn. Xem mục I.9 để biết vì sao không có tỉ lệ nào giữa hai ngôn ngữ để nhân.

Tự chấm từ thư mục gốc của nhánh Python:

```
python3 chay.py topics/01-nen-tang-cpp/solutions/bai01.py
python3 chay.py topics/01-nen-tang-cpp --tat-ca
```

Bài	Tên	Kỹ thuật chính	Cạm bẫy Python	File nộp
A	Tổng và tích lớn nhất	xét đủ trường hợp	cho số nguyên đi qua float	bai01.py
B	Hạng của phần tử	nén toạ độ	bisect từng phần tử thay vì một dict	bai02.py
C	Đếm cặp có tổng cho trước	bảng tần suất một lượt	d[k] ném KeyError, defaultdict thì tạo khoá	bai03.py
D	Truy vấn đếm không vượt quá	tìm kiếm nhị phân	bisect_left thay vì bisect_right	bai04.py
E	Sắp xếp phân số	biến thứ tự thành khoá	double gộp hai phân số, và cmp_to_key chậm	bai05.py
F	Gộp đồng chi phí nhỏ nhất	heapq	n lần heappush thay vì một heapify	bai06.py
G	Bảng xếp hạng	khoá nhiều tầng	tầng giảm dần không đảo dấu được cho xâu	bai07.py
H	Vòng tròn	dấu của %	tướng phải chuẩn hoá như C++, rồi chuẩn hoá sai	bai08.py
I	Đếm từ trong câu	dòng khác token	read().split() xoá mất cấu trúc dòng	bai09.py
J	Người bạn gần nhất	bisect trên list đã sắp	Python không có tập hợp có thứ tự	bai10.py

Bạn viết bài của mình vào solutions/bai01.py; bản mẫu nằm riêng ở solutions/ref/.

Bài A. Tổng và tích lớn nhất

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên. Hãy in ra hai giá trị: tổng của cả dãy, và tích lớn nhất có thể thu được khi nhân hai phần tử ở hai vị trí khác nhau.

Dữ liệu vào

- Dòng 1: số nguyên n ($2 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên $a_1 \dots a_n$ ($|a_i| \leq 10^9$)

Kết quả

Một dòng gồm hai số cách nhau bởi dấu cách: tổng dãy và tích lớn nhất.

Ví dụ

Dữ liệu vào	Kết quả
5 3 -1 -7 2 4	1 12

Giải thích

Tổng bằng 1. Tích lớn nhất là $3 \cdot 4 = 12$.

Lưu ý

Hãy thử dãy $-10^{-9} \ 1 \ 2$ trước khi tin rằng đáp án luôn nằm ở hai phần tử lớn nhất. Và ở Python, đừng để một con số nào của bài này đi qua float: đáp án lên tới 10^{18} còn float mất chính xác từ $9 \cdot 10^{15}$.

Bài B. Hạng của phần tử

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên. Với mỗi phần tử, hãy in ra hạng của nó, tức là vị trí của giá trị đó trong danh sách các **giá trị phân biệt** đã sắp xếp tăng dần, đánh số từ 1.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)

Kết quả

Một dòng gồm n số, hạng tương ứng với từng phần tử theo đúng thứ tự ban đầu.

Ví dụ

Dữ liệu vào	Kết quả
6 50 10 50 20 10 30	4 1 4 2 1 3

Giải thích

Các giá trị phân biệt sắp tăng là 10, 20, 30, 50 nên hạng lần lượt là 1, 2, 3, 4.

Lưu ý

Kỹ thuật của bài này tên là nén toạ độ, và bạn sẽ dùng lại nó trong hầu hết bài Fenwick tree hay segment tree về sau. Ở Python nó gần tới mức khó tin; cái đáng nghĩ là bước tra hạng, vì có hai cách khác nhau một bậc độ phức tạp.

Bài C. Đếm cặp có tổng cho trước

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và một số X . Đếm số cặp chỉ số (i, j) với $i < j$ sao cho $a_i + a_j = X$.

Dữ liệu vào

- Dòng 1: n và X ($1 \leq n \leq 2 \cdot 10^5, |X| \leq 2 \cdot 10^9$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)

Kết quả

Một số nguyên duy nhất là số cặp tìm được.

Ví dụ

Dữ liệu vào	Kết quả
6 7 1 6 3 4 3 2	3

Giải thích

Các cặp là $(1, 6)$, rồi $(3, 4)$ ở vị trí 3 và 4, rồi $(3, 4)$ ở vị trí 5 và 4.

Lưu ý

Với n bằng $2 \cdot 10^5$ và mọi phần tử giống nhau, đáp án cỡ $2 \cdot 10^{10}$. Ở C++ đó là một cái bẫy tràn số; ở Python nó không là gì cả. Hãy tự hỏi cái gì đã thay thế nó, và câu trả lời nằm ở cách bạn đọc một khoá chưa có trong từ điển.

Bài D. Truy vấn đếm không vượt quá

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và q truy vấn. Mỗi truy vấn cho một số x , hãy đếm xem có bao nhiêu phần tử của dãy có giá trị nhỏ hơn hoặc bằng x .

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- Dòng 3: q ($1 \leq q \leq 2 \cdot 10^5$)
- Dòng 4: q số nguyên $x_1 \dots x_q$ ($|x_i| \leq 10^9$)

Kết quả

q dòng, mỗi dòng một đáp án.

Ví dụ

Dữ liệu vào	Kết quả
5	0
5 1 4 1 3	2
4	3
0 1 3 100	5

Giải thích

Với $x = 1$ có hai phần tử thoả mãn, đó là hai số 1. Với $x = 3$ có thêm số 3 nên đáp án là 3.

Lưu ý

Dãy không đổi trong suốt quá trình truy vấn, nên bạn được phép tiền xử lý một lần rồi trả lời mỗi truy vấn trong $O(\log n)$. Đây là một trong hai bài duy nhất của buổi mà phạm bẫy của C++ còn nguyên vẹn ở Python, chỉ đổi tên hàm.

Bài E. Sắp xếp phân số

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho n phân số dương, phân số thứ i là a_i / b_i . Hãy sắp xếp chúng tăng dần theo giá trị. Hai phân số có giá trị bằng nhau thì phân số có mẫu số nhỏ hơn đứng trước.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- n dòng tiếp theo, dòng thứ i gồm a_i và b_i ($1 \leq a_i, b_i \leq 10^9$)

Kết quả

n dòng, mỗi dòng là một phân số theo thứ tự đã sắp, giữ nguyên dạng $a_i \ b_i$ như dữ liệu vào.

Ví dụ

Dữ liệu vào	Kết quả
4	1 3
1 2	1 2
1 3	2 4
2 4	3 4
3 4	

Giải thích

Giá trị của bốn phân số lần lượt là 0,5 rồi một phần ba rồi 0,5 rồi 0,75. Hai phân số $1/2$ và $2/4$ bằng nhau nên phân số có mẫu số 2 đứng trước.

Lưu ý

Bài này cấm dùng số thực, và lệnh cấm ấy có rằng. `float` của Python cũng chỉ là `double` 53 bit, trong khi hai phân số với tử và mẫu cỡ 10^9 có thể khác nhau ở chữ số thứ 18. Test 04 đúng là ca ấy. Câu hỏi thật của bài: Python sắp xếp bằng **khóa**, vậy làm sao biến "giá trị đúng của a/b " thành một khóa sắp xếp được?

Bài F. Gộp đồng chi phí nhỏ nhất

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n đồng sỏi, đồng thứ i nặng a_i . Mỗi bước bạn chọn hai đồng bất kỳ và gộp chúng lại thành một đồng mới, chi phí của bước đó bằng tổng khối lượng hai đồng vừa gộp. Lặp lại cho tới khi chỉ còn một đồng. Hãy tìm tổng chi phí nhỏ nhất.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất là tổng chi phí nhỏ nhất. Với n bằng 1 thì không cần gộp lần nào, chi phí bằng 0.

Ví dụ

Dữ liệu vào	Kết quả
4 1 2 3 4	19

Giải thích

Gộp 1 với 2 hết 3, được đồng 3. Gộp 3 với 3 hết 6, được đồng 6. Gộp 6 với 4 hết 10. Tổng là 19. Mọi thứ tự gộp khác đều tổn không ít hơn.

Lưu ý

Trước khi tin vào chiến lược tham lam của mình, hãy tự hỏi: một đồng bị đem đi gộp càng nhiều lần thì khối lượng của nó được cộng vào tổng chi phí bao nhiêu lần? Và hãy kiểm tra $n = 1$, vì đó là chỗ một vòng lặp viết hơi rộng tay sẽ lấy phần tử từ một đồng rỗng.

Bài G. Bảng xếp hạng

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Sau một kỳ thi có n thí sinh, mỗi thí sinh có một tên và một điểm số. Hãy in ra k thí sinh dẫn đầu. Thứ tự xếp hạng: điểm cao đứng trước, hai thí sinh bằng điểm thì tên nào nhỏ hơn theo thứ tự từ điển đứng trước.

Dữ liệu vào

- Dòng 1: n và k ($1 \leq k \leq n \leq 2 \cdot 10^5$)
- n dòng tiếp theo, mỗi dòng gồm một tên và một điểm. Tên chỉ gồm chữ cái Latin thường, dài không quá 20 ký tự, và không có hai thí sinh trùng tên. Điểm là số nguyên từ 0 tới 10^9 .

Kết quả

k dòng, mỗi dòng gồm tên và điểm của một thí sinh theo thứ tự xếp hạng.

Ví dụ

Dữ liệu vào	Kết quả
5 3 an 90 binh 95 cuong 90 dung 70 em 95	binh 95 em 95 an 90

Giải thích

Hai thí sinh cùng đạt 95 điểm, trong đó binh đứng trước em theo thứ tự từ điển.

Lưu ý

Hai tầng thứ tự đi **ngược chiều nhau**, và đó là toàn bộ cái khó ở Python: sort nhận khoá chứ không nhận hàm so sánh. Hãy nghĩ trước xem tầng nào đảo dấu được và tầng nào không.

Bài H. Vòng tròn

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n ô xếp thành vòng tròn, đánh số từ 0 tới $n - 1$ theo chiều kim đồng hồ. Một con trỏ ban đầu ở ô 0. Có q lệnh, lệnh thứ i cho số nguyên d_i nghĩa là dịch con trỏ đi d_i ô theo chiều kim đồng hồ, giá trị âm nghĩa là dịch ngược chiều. Sau mỗi lệnh, hãy in vị trí hiện tại của con trỏ.

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n \leq 10^9$, $1 \leq q \leq 2 \cdot 10^5$)
- Dòng 2: q số nguyên $d_1 \dots d_q$ ($|d_i| \leq 10^{18}$)

Kết quả

q dòng, mỗi dòng là vị trí con trỏ sau lệnh tương ứng.

Ví dụ

Dữ liệu vào	Kết quả
5 4	3
3 -4 10 -1	4
	4
	3

Giải thích

Từ ô 0 đi 3 bước tới ô 3. Lùi 4 bước từ ô 3 thì vòng qua ô 0 và dừng ở ô 4. Đi tiếp 10 bước là đúng hai vòng nên vẫn ở ô 4. Lùi 1 bước về ô 3.

Lưu ý

Bản C++ của bài này có hai cái bẫy. **Cả hai đều không tồn tại ở Python**, và đây là bài duy nhất của buổi như vậy. Đừng chép công thức chuẩn hoá của C++ sang: nó không sai, nó chỉ là mã chết, và viết mã chết mà không biết vì sao nó chết là đã hiểu sai ngôn ngữ mình đang dùng.

Bài I. Đếm từ trong câu

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho n dòng văn bản. Mỗi dòng gồm các từ viết bằng chữ cái Latin, ngăn cách nhau bởi một hoặc nhiều dấu cách, và có thể có dấu cách thừa ở đầu hoặc cuối dòng. Với mỗi dòng, hãy in ra số từ và từ dài nhất của dòng đó. Nếu có nhiều từ cùng đạt độ dài lớn nhất thì in từ xuất hiện sớm nhất. Nếu dòng không có từ nào, hãy in 0 -.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 10^5$)
- n dòng tiếp theo, mỗi dòng dài không quá 10^5 ký tự, tổng độ dài không quá 10^6 .

Kết quả

n dòng, mỗi dòng gồm số từ và từ dài nhất, cách nhau bởi một dấu cách.

Ví dụ

Dữ liệu vào	Kết quả
3 the quick brown fox a bb ccc hello world	4 quick 3 ccc 2 hello

Giải thích

Ở dòng thứ ba, hai từ hello và world cùng dài 5 ký tự nên kết quả lấy từ đứng trước.

Lưu ý

Đây là bài về **dòng**, không phải về token, nên khung `read().split()` của kho này là sai về bản chất chứ không phải chậm. Hai chuyện mà C++ phải làm thủ công thì Python cho không, và đúng một chuyện thì ngược lại. Hãy tìm ra cả ba trước khi gõ.

Bài J. Người bạn gần nhất

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Ban đầu bạn có một tập hợp rỗng. Có q thao tác, mỗi thao tác thuộc một trong hai loại:

- 1 x : thêm số x vào tập hợp. Nếu x đã có sẵn thì tập hợp không đổi.
- 2 x : in ra khoảng cách nhỏ nhất từ x tới một phần tử của tập hợp, tức là giá trị nhỏ nhất của $|y - x|$ với y thuộc tập hợp. Nếu tập hợp đang rỗng thì in -1 .

Dữ liệu vào

- Dòng 1: q ($1 \leq q \leq 2 \cdot 10^5$)
- q dòng tiếp theo, mỗi dòng gồm loại thao tác và giá trị x ($|x| \leq 10^9$)

Kết quả

Mỗi thao tác loại 2 in ra một dòng đáp án.

Ví dụ

Dữ liệu vào	Kết quả
6	- 1
2 5	2
1 10	1
1 3	
2 5	
1 4	
2 5	

Giải thích

Truy vấn đầu tiên rơi vào lúc tập hợp còn rỗng. Ở truy vấn thứ hai, tập hợp là $3, 10$ và số gần 5 nhất là 3. Ở truy vấn cuối, tập hợp là $3, 4, 10$ và số gần 5 nhất là 4.

Lưu ý

Phần tử gần x nhất chỉ có thể là một trong hai phần tử kề x trong thứ tự sắp xếp. Cả hai đều có thể không tồn tại, và đó chính là chỗ bài này đánh gục phần lớn lời giải. Thêm một câu hỏi chỉ có ở Python: `set` của Python **không có thứ tự**, vậy bạn giữ tập hợp bằng cái gì, và cái đó tốn bao nhiêu mỗi lần thêm?

Phần III. Gợi ý và lời giải

Đọc từng tầng một, dừng lại ngay khi đã nghĩ ra. Code đầy đủ nằm ở Phần IV.

Bài	Cái bẫy thật sự ở Python
A	tích lớn nhất có thể là hai số âm; và mọi con số phải ở lại trong int
B	tra hạng bằng bisect từng phần tử là $O(n \log n)$ khi $O(n)$ là đủ
C	d[k] ném KeyError, còn defaultdict[k] thì lặng lẽ tạo khoá
D	bisect_left thay vì bisect_right, sai ngay trên ví dụ của đề
E	biến thứ tự thành khoá; cmp_to_key đúng nhưng chậm gấp đôi
F	heapq chỉ có min-heap; heapij thay cho n lần heappush
G	xâu không đảo dấu được, nên tầng giảm dần phải là tăng số
H	không có bẫy nào cả, và đó chính là bài học
I	read().split() xóa mất cấu trúc dòng; split(b" ") đếm cả mảnh rỗng
J	Python không có tập hợp có thứ tự; insert trả giá bằng memmove

Bài A. Tổng và tích lớn nhất

Gợi ý 1. Ở C++ bài này chủ yếu là bài tập về kiểu dữ liệu: tổng đạt $2 \cdot 10^{14}$ và tích đạt 10^{18} . Ở Python cả hai con số ấy không là gì cả. Vậy còn lại gì?

Gợi ý 2. Còn lại phần chưa bao giờ về kiểu dữ liệu. Hãy thử dãy $-10^{-9} 1^2$ bằng tay.

Gợi ý 3. Sau khi sắp xếp, chỉ có đúng hai ứng viên: $a[n-1] * a[n-2]$ và $a[0] * a[1]$.

Lời giải. sorted, rồi lấy giá trị lớn hơn trong hai ứng viên, rồi in cùng sum(a). Cả sum lẫn sorted đều là vòng lặp ở mức C.

Bẫy của C++ là gì và vì sao nó biến mất. Bên C++, int a[] rồi long long p = a[i] * a[j]; vẫn tràn, vì phép nhân đã tính xong bằng int trước khi có phép gán nào. Python không có độ rộng cố định nên chuyện đó không xảy ra được.

Cái thay thế nó, và nó ở đúng chỗ ấy. Con đường duy nhất làm hỏng bài này ở Python là để một giá trị đi qua float. Tôi đo: int(float(999999937 * 999999937)) lệch **1 đơn vị** so với tích đúng, và int((10**18 + 1) / 1) cũng lệch 1. Tức là chỉ cần một dấu / lạc vào, hoặc một sum của các giá trị đã bị float hoá, là bạn mất bài. Và hãy chú ý cái bẫy con: float(10**18) == 10**18 cho True, nên thử đúng một giá trị rồi yên tâm là cách chắc chắn nhất để tin nhầm.

Một chi tiết nhỏ về max. Viết max(a[-1] * a[-2], a[0] * a[1]) đúng và ngắn. Bản mẫu viết bằng if vì trong một vòng lặp nóng thì một lời gọi max hai tham số đắt hơn một phép so sánh; ở đây nó chạy đúng một lần nên khác biệt bằng không, và cả hai cách đều tốt.

Bài B. Hạng của phần tử

Gợi ý 1. Hạng của $a[i]$ chính là số lượng giá trị phân biệt nhỏ hơn $a[i]$, cộng một.

Gợi ý 2. Bốn bước của C++ là sort, unique, erase, lower_bound. Ba bước đầu ở Python gộp lại thành mấy ký tự?

Gợi ý 3. `sorted(set(a))`. Bước thứ tư thì còn hai lựa chọn, và chúng khác nhau một bậc độ phức tạp.

Lời giải. `sorted(set(a))` cho danh sách giá trị phân biệt đã sắp. Sau đó dựng một dict từ giá trị sang hạng bằng một comprehension, rồi tra từng phần tử. Tổng chi phí $O(n \log n)$ cho lần sắp, cộng $O(n)$ cho n phép tra.

Hai bẫy của C++ biến mất cùng một lúc. unique chỉ khử phần tử trùng **liền kề** nên quên sort là hỏng; và unique không thu nhỏ vector nên quên erase là đuôi còn rác. Cả hai lỗi tồn tại được vì quy trình có bốn bước tách rời. `sorted(set(a))` là **một** lời gọi lồng một lời gọi, không có thứ tự nào để viết sai.

Cái thay thế nó là một lựa chọn về chi phí. Hai cách tra hạng:

- `bisect_left(b, x) + 1` cho mỗi phần tử: n phép tìm kiếm nhị phân, $O(n \log n)$.
- dựng một dict một lần rồi `hang[x]: 0(n)`.

Vì **mọi** phần tử đều phải tra, dict luôn thắng ở bài này. `bisect` chỉ hơn khi bạn chỉ cần tra vài giá trị, hoặc khi cần câu hỏi "giá trị nào không có trong bảng thì nằm ở đâu", mà bài này không cần.

Bài C. Đếm cặp có tổng cho trước

Gợi ý 1. Duyệt mọi cặp là $4 \cdot 10^{10}$ phép. Không được, ở ngôn ngữ nào cũng vậy. Vậy phải cố định một đầu và tìm nhanh đầu còn lại.

Gợi ý 2. Khi đứng ở vị trí j , bạn cần biết có bao nhiêu vị trí $i < j$ mà $a[i]$ bằng đúng $X - a[j]$.

Gợi ý 3. Một bảng tần suất của những phần tử **đã đi qua**. Cộng dồn trước rồi mới thêm $a[j]$ vào bảng.

Lời giải. Một lượt quét. Mỗi bước cộng số lần đã thấy $X - v$ vào đáp án, rồi tăng số lần đã thấy v . Điều kiện $i < j$ được bảo đảm miễn phí, và các giá trị trùng nhau xử lý đúng mà không cần chia trường hợp.

Ba bẫy của C++, cả ba đều biến mất. Đáp án $2 \cdot 10^{10}$ và X cỡ $2 \cdot 10^9$ đều tràn int, còn `unordered_map` thì bị hack bằng test gây đống độ. Python không có độ rộng cố định, và dict là bảng băm duy nhất của nó nên không có lựa chọn nào để chọn sai.

Cái thay thế chúng là một bẫy im lặng của chính dict. Ba cách viết, ba hành vi:

Cách viết	Khi khoá chưa có
<code>d[k]</code> trên dict	ném <code>KeyError</code> , tức RE trên máy chấm
<code>d.get(k, 0)</code>	trả 0, không đổi d. Đây là cách đúng
<code>d[k]</code> trên <code>collections.defaultdict(int)</code>	trả 0 và tạo luôn khoá

Dòng thứ ba đúng là cái bẫy mà bản C++ cảnh báo cho `std::map: m[key]` tự tạo phần tử. Nó không biến mất, nó chỉ chuyển nhà từ map sang `defaultdict`. Ở bài này mỗi phần tử sinh ra tối đa một khoá thừa, nên hậu quả là bộ nhớ gấp đôi chứ không phải kết quả sai. Nhưng trong một bài đếm khác, nơi bạn lặp lại `if d[k] > 0` trên nhiều khoá lạ, `defaultdict` làm từ điển phình lên tới hết bộ nhớ.

Một chi tiết vật đáng giữ. Bản mẫu buộc `da_thay.get` vào một tên cục bộ trước vòng lặp. Xem mục I.3: tra tên cục bộ đi bằng chỉ số, tra thuộc tính đi bằng từ điển, và vòng lặp này chạy $2 \cdot 10^5$ lượt.

Bài D. Truy vấn đếm không vượt quá

Gợi ý 1. Dãy không đổi suốt quá trình truy vấn, nên được phép tiền xử lý một lần.

Gợi ý 2. Nếu dãy đã sắp, số phần tử nhỏ hơn hoặc bằng x chính là vị trí của phần tử đầu tiên **lớn hơn thật sự** x .

Gợi ý 3. Đó đúng là định nghĩa của `bisect_right`, chứ không phải `bisect_left`.

Lời giải. Sắp một lần $O(n \log n)$, mỗi truy vấn một lần `bisect_right` là $O(\log n)$.

Đây là một trong hai bài mà cạm bẫy của C++ sống nguyên vẹn, vì nó không phải bẫy của ngôn ngữ mà là bẫy của việc đọc đề. Bảng tương ứng:

Cần đếm	C++	Python
số phần tử $< x$	<code>lower_bound(x) - begin()</code>	<code>bisect_left(a, x)</code>
số phần tử $\leq x$	<code>upper_bound(x) - begin()</code>	<code>bisect_right(a, x)</code>
số phần tử $= x$	<code>upper_bound - lower_bound</code>	<code>bisect_right(a, x) - bisect_left(a, x)</code>
số phần tử trong $[l, r]$	<code>upper_bound(r) - lower_bound(l)</code>	<code>bisect_right(a, r) - bisect_left(a, l)</code>

Chọn nhầm hàng đầu thay hàng hai thì sai **trên ví dụ của đề**: truy vấn $x = 1$ gặp hai số 1 trong dãy, `bisect_left` trả 0 còn đáp án là 2.

Phần Python thuần túy của bài này nằm ở vòng lặp, không ở phép tìm. `bisect_right` là code C nên $2 \cdot 10^5$ lời gọi gần như miễn phí. Nhưng tra tên `bisect.bisect_right` qua module mỗi lượt thì không miễn phí, và gọi `print()` mỗi truy vấn cũng vậy. Buộc hàm vào tên cục bộ, gom kết quả, ghi một lần. Xem số đo ở mục I.3.

Bài E. Sắp xếp phân số

Gợi ý 1. Đề cấm số thực, và lệnh cấm ấy đúng: `float` của Python cũng là `double` 53 bit.

Gợi ý 2. C++ giải bằng comparator nhân chéo: $a/b < c/d$ tương đương $a*d < c*b$ khi mẫu dương. Python thì `sort` nhận **khóa**, không nhận comparator. Bạn có ba lựa chọn, và chúng không tương đương về giá.

Gợi ý 3. Lựa chọn tốt nhất là biến giá trị đúng của a/b thành một **số nguyên** sắp xếp được. Nhân lên rồi lấy phần nguyên. Nhân lên bao nhiêu thì đủ?

Lời giải. Dùng khóa $(a * 10^{20}) // b$, kèm mẫu số làm tăng hai để xử lý luật hoà.

Vì sao 10^{20} là đủ, và đây là chỗ phải chứng minh chứ không phải chọn bừa. Hai phân số **khác nhau** với tử mẫu tối 10^9 chênh nhau ít nhất $1/(b_1 \cdot b_2)$, tức ít nhất 10^{-18} . Bước lượng tử của khóa là 10^{-20} . Vậy hai giá trị khác nhau cách nhau ít nhất **100 bước**, nên phép lấy phần nguyên không thể gộp chúng lại. Hai phân số **bằng nhau** thì cùng một số thực nên cho cùng khóa. Đúng hai điều đó là tất cả những gì cần.

Tôi kiểm lại bằng máy thay vì tin lập luận: đối chiếu khóa với `fractions.Fraction` trên $2 \cdot 10^5$ cặp ngẫu nhiên, cộng thêm các cặp Farey kề nhau dựng riêng để ép sát ngưỡng. **Không một ca lệch.** Trên chính cặp của test 04, hai khóa cách nhau 861 đơn vị, dư dả so với mức 1 đơn vị cần thiết.

Vì sao `float` chết ở đây, bằng số của test 04. Hai phân số $433494437/267914296$ và $701408733/433494437$ đều cho 1.618033988749895 khi chia bằng `float`, và $x/y == p/q$ trả về **True**. Nhân chéo đúng thì chênh nhau đúng 1. Khi `float` gộp hai giá trị làm một, luật phá hoà theo mẫu số cho ra thứ tự ngược, nên lỗi lộ ra thành WA chứ không phải thành một thứ tự chỉ hơi lệch.

Ba cách đều đúng, giá khác nhau. Đo trên $2 \cdot 10^5$ phân số ngẫu nhiên: khóa số nguyên **0,210 s**, `functools.cmp_to_key` **0,443 s**, `fractions.Fraction` làm khóa **1,690 s**. Lý do `cmp_to_key` chậm không phải là comparator xấu, mà là số lời gọi: khóa gọi n lần, comparator gọi khoảng $n \log n$ lần, và mỗi lần là một lời gọi Python chứ không phải một phép so sánh ở mức C. Lý do `Fraction` chậm hơn nữa là mỗi phép so sánh của nó lại là một lời gọi Python có rút gọn phân số bên trong.

Và bẫy lớn nhất của C++ thì không tồn tại. Luật comparator chặt, thứ làm `sort` của `libstdc++` đọc ra ngoài mảng và sắp chỉ khi mảng đủ lớn, không có ở Python. Không có comparator thì không có hợp đồng để vi phạm. Đây là một trong những chỗ Python an toàn hơn thật sự, không phải chỉ dễ viết hơn.

Bài F. Gộp đồng chi phí nhỏ nhất

Gợi ý 1. Một đồng bị gộp càng nhiều lần thì khối lượng của nó bị tính vào tổng chi phí càng nhiều lần. Vậy đồng nặng nên bị gộp sớm hay muộn?

Gợi ý 2. Ở mỗi bước, chọn hai đồng nhẹ nhất hiện có. Đó là tham lam, và nó đúng: tổng chi phí bằng tổng $a_i \cdot d_i$ với d_i là độ sâu của lá i trong cây gộp, tức đúng bài toán mã Huffman.

Gợi ý 3. Bạn cần một cấu trúc luôn cho ra phần tử nhỏ nhất và nhận thêm phần tử mới.

Lời giải. heapq. Lấy ra hai phần tử nhỏ nhất, cộng chi phí, đẩy tổng trở lại, cho tới khi còn một phần tử. $O(n \log n)$.

Bẫy của C++ biến mất, và cái thay thế là hình ảnh trong gương của nó. priority_queue mặc định là max-heap, nên muốn min-heap phải viết đủ ba tham số priority_queue<ll, vector<ll>, greater<ll>>; viết thiếu thì chương trình vẫn chạy, vẫn in một con số, chỉ là con số sai. heapq thì ngược lại: nó **chỉ có** min-heap và không có tham số nào để viết sai. Ở bài này Python cho không. Nhưng khi bạn cần một max-heap, heapq không có, và cách làm chuẩn là đẩy giá trị **đã đảo dấu** rồi đảo lại khi lấy ra. Đó là chỗ duy nhất heapq gây lỗi, và nó gây lỗi ở bài khác chứ không ở bài này.

Một mẹo Python về chi phí. heapify(danh_sach) dựng đồng tại chỗ trong $O(n)$ bằng một vòng lặp ở mức C . n lần heappush cho cùng kết quả nhưng là n lời gọi từ Python và $O(n \log n)$. Bản mẫu dùng heapify.

Ca n = 1 phải in 0. while len(h) > 1 xử lý sẵn. Viết while h thì lần heappop thứ hai lấy từ một đồng rỗng và ném IndexError, tức RE. Đây là ca biên duy nhất của bài, và test 02 chính là nó.

Bài G. Bảng xếp hạng

Gợi ý 1. Chỉ là một lần sắp xếp với hai tầng thứ tự.

Gợi ý 2. Hai tầng đi ngược chiều nhau: điểm **giảm**, tên **tăng**. Ở C++ đó là một comparator hai tầng. Ở Python sort nhận khoá, và khoá thì không có chiều.

Gợi ý 3. Số thì đảo dấu được. Xâu thì không.

Lời giải. key là tuple (-diem, ten), rồi lấy k dòng đầu. Tuple so sánh từ điển nên hai tầng tự xếp đúng, và dấu trừ biến tăng điểm thành giảm dần.

Bẫy của C++ biến mất hẳn. Comparator hai tầng của C++ có một hình dạng bắt buộc: mọi tầng trừ tầng cuối phải kiểm **khác nhau** trước rồi mới so sánh, còn tầng cuối phải dùng < chặt. Viết if (a.score > b.score) return true; rồi return a.name < b.name; là sai, vì khi a.score < b.score nó vẫn rơi xuống dòng so tên. Với khoá tuple thì không có hình dạng nào để viết sai: các tầng nằm cạnh nhau trong một tuple và Python lo phần còn lại.

Cái thay thế nó là câu hỏi "tầng này đảo được không". Nếu đề đổi thành "điểm giảm dần, tên **giảm** dần" thì dấu trừ không dùng được cho tên. Cách đúng lúc đó là sắp hai lần và dựa vào tính **ổn định** của sort: sắp theo tầng phụ trước, rồi sắp lại theo tầng chính. Cách sai là key=lambda x: (-x.diem, -ord(x.ten[0])), vì nó chỉ so được ký tự đầu.

Một chi tiết vào ra đáng giữ. Tên lấy thẳng từ split() nên chúng là **bytes**. So sánh bytes cho đúng thứ tự từ điển như so sánh xâu ASCII, và giữ nguyên bytes bỏ được $2 \cdot 10^5$ lời gọi giải mã; chỉ k dòng thật sự in ra mới cần decode().

Bài H. Vòng tròn

Gợi ý 1. Vị trí mới là vị trí cũ cộng d, rồi lấy phần dư cho n.

Gợi ý 2. Ở C++ có hai vấn đề với câu trên. Hãy thử in -1 % 5 trong Python rồi tự trả lời còn vấn đề nào.

Gợi ý 3. Không còn vấn đề nào.

Lời giải. vt = (vt + d) % n, in ra. Hết bài. $O(q)$.

Đây là bài duy nhất của buổi mà Python dễ hơn hẳn, và nó dễ hơn vì hai lý do riêng biệt.

Lý do thứ nhất: phần dư của Python mang **dấu của số chia**, nên với $n > 0$ thì $x \% n$ luôn nằm trong $[0, n)$ sẵn. $-1 \% 5$ cho 4. Công thức $((x \% n) + n) \% n$ của C++ ở đây là mã chết. Chép nó sang không sai kết quả, nhưng nó tố cáo rằng bạn chưa hiểu mình đang dùng ngôn ngữ nào.

Lý do thứ hai: bản C++ phải rút $d \% n$ **trước** khi cộng, vì cộng dồn $2 \cdot 10^5$ bước mỗi bước 10^{18} cho $2 \cdot 10^{23}$ và tràn long long từ lâu. Ở Python số nguyên vô hạn nên cộng dồn tất rồi lấy dư một lần cũng chính xác tuyệt đối.

Nhưng đừng rút ra bài học sai. Cái làm bài này dễ, tức `//` và `%` làm tròn về **âm vô cùng**, chính là cái làm bài khác khó. Xem mục I.1: $-7 // 2$ cho -4 ở Python và -3 ở C++. Khi bạn cần "cắt về không" như C++, `//` trả lời sai một đơn vị với số âm, và cách chữa là `-( -a // b)` chứ không phải `int(a / b)`, vì cái sau đi qua float.

Một chi tiết về chi phí, không phải về đúng sai. Bản mẫu vẫn giữ vt đã rút gọn sau mỗi bước, dù không bắt buộc. Lý do: để một biến tích lũy phình quá $2 \cdot 10^{23}$ thì mọi phép cộng sau đó là số học nhiều từ máy, đắt hơn mà chẳng được gì.

Bài I. Đếm từ trong câu

Gợi ý 1. Bài này về **dòng**. Khung `read().split()` của kho này gộp mọi dòng thành một dãy token và xoá sạch ranh giới dòng.

Gợi ý 2. Vậy hãy cắt theo đúng một thứ: ký tự xuống dòng.

Gợi ý 3. Còn tách từ **trong** một dòng thì dùng `split()` không đối số. Hãy tự hỏi nó khác `split(b" ")` ở chỗ nào, vì đề nói rõ có dấu cách thừa và dấu cách lặp.

Lời giải. Đọc cả khối, cắt theo `b"\n"`, phần tử 0 là n, n phần tử tiếp là các dòng. Với mỗi dòng, `split()` cho danh sách từ; `len` cho số từ; `max(tu, key=len)` cho từ dài nhất và **trả về cái xuất hiện sớm nhất** trong các cái cùng dài nhất, đúng luật hoà của đề.

Bẫy của C++ là `cin.ignore()`: sau `cin >> n` ký tự xuống dòng vẫn nằm trong bộ đệm nên `getline` đầu tiên trả về xâu rỗng, mọi dòng sau lệch đi một, và bạn mất trắng bài dù thuật toán chẳng có gì.

Bẫy của Python có cùng hình dạng, hai nguyên nhân khác. Một là `sys.stdin.readline()` **giữ lại** ký tự xuống dòng, nên độ dài lớn hơn thật một đơn vị. Hai, và đây mới là cái hay gặp, là dùng `read().split()` theo phản xạ: nó đúng cho chín bài của buổi này và sai cho đúng bài này.

Hai thứ Python cho không, và đúng một thứ thu lại.

- Cho không thứ nhất: C++ cần `istreamstream` để bỏ qua dấu cách thừa, dấu cách lặp và dấu cách hai đầu. `split()` không đối số của Python làm cả ba sẵn. `b" a bb "` cho đúng `[b"a", b"bb"]`.
- Cho không thứ hai: C++ phải so `w.size() > best.size()` với dấu **lớn hơn thật sự** để từ đầu tiên thắng khi hoà. `max(..., key=len)` của Python đã trả về cái sớm nhất, nên luật hoà đúng miễn phí.
- Thu lại: `split(b" ")` **có** đối số không gộp gì cả. Với `b" a bb "` nó cho `[b"", b"a", b"", b"bb", b""]`, tức 5 từ thay vì 2. Một dấu cách trong ngoặc là toàn bộ khác biệt.

Ca biên là dòng không có từ nào, phải in 0 -, và test 09 có những dòng như vậy. Nếu viết `max(tu, key=len)` mà không kiểm tra thì nó ném `ValueError` trên dãy rỗng, tức đổi một WA thành một RE.

Bài J. Người bạn gần nhất

Gợi ý 1. Nếu tập hợp được giữ ở dạng đã sắp, phần tử gần x nhất chỉ có thể là một trong hai phần tử kề x trong thứ tự đó.

Gợi ý 2. C++ dùng `std::set`, vốn là cây cân bằng nên vừa có thứ tự vừa có `lower_bound`. `set` của Python là bảng băm và **không có thứ tự**. Vậy dùng gì?

Gợi ý 3. Một list giữ cho luôn sắp, cộng `bisect_left`. Ứng viên thứ nhất là phần tử ở vị trí tìm được, ứng viên thứ hai là phần tử ngay trước nó. Đừng quên hai ca biên và ca tập rỗng.

Lời giải. `bisect_left(ds, x)` cho vị trí `i`. Nếu `i == len(ds)` thì chỉ có ứng viên bên trái; nếu `i == 0` thì chỉ có ứng viên bên phải; còn lại thì lấy cái nhỏ hơn trong hai hiệu. Tập rỗng thì in `-1`. Thêm vào thì chèn tại `i`, và chỉ chèn khi `x` chưa có.

Bốn cái bẫy của C++, và số phận từng cái ở Python.

Bẫy C++	Ở Python
<code>std::lower_bound</code> tự do chạy tuyến tính trên iterator của cây	biến mất , <code>bisect</code> chỉ làm việc trên list có chỉ số ngẫu nhiên
giải tham chiếu <code>it == s.end()</code> là lỗi bộ nhớ	thành <code>i == len(ds)</code> , và lỡ tay thì <code>IndexError</code> có truy vết chứ không phải lỗi im lặng
gọi <code>--it</code> khi <code>it == s.begin()</code>	thành <code>i == 0</code> , và <code>ds[-1]</code> chạy được và cho phần tử cuối, tức một WA lặng lẽ. Đây là ca xấu hơn C++
tập rỗng phải in <code>-1</code>	y hết , và <code>if not ds</code> bắt được cả hai ca biên cùng lúc

Dòng thứ ba đáng dừng lại. Ở C++, `--it` từ `begin()` là hành vi không xác định, thường sập. Ở Python, `ds[i - 1]` với `i = 0` là `ds[-1]`, tức phần tử **cuối** dãy, và chương trình chạy tiếp ngoan ngoãn với một đáp án sai. Chỉ số âm của Python là một tiện nghi, và ở đây nó biến một lỗi ồn ào thành một lỗi câm.

Cái thay thế lớn nhất là chi phí. set của C++ chèn trong $O(\log n)$. list cộng `bisect` tìm trong $O(\log n)$ nhưng **chèn** phải dồn đuôi mảng, tức $O(n)$ phần tử được chép. Đo trên máy này: 10^5 lần chèn có sắp thứ tự mất **0,625 s**, trong khi sắp cùng ngàn ấy giá trị một lần chỉ mất **0,038 s**, tức đắt gấp **16 lần**. Cả bài J trên test 09 chạy **0,707 s**, tức 24 phần trăm giới hạn 3 giây.

Nó kịp ở đây vì phép dồn là một lệnh `memmove` ở mức C trên một mảng còn nằm trong cache. Nó sẽ **không** kịp nếu `q` lớn hơn một bậc, hoặc nếu cùng cách làm ấy nằm bên trong một thuật toán đã tốn $O(n \log n)$.

Bài học đúng không phải "tránh `bisect.insort`". Bài học là: khi mọi giá trị đã biết trước, hãy **sắp một lần** rồi làm việc trên dãy tĩnh, rẻ hơn 16 lần. Ở bài này không làm được, vì truy vấn xen kẽ với phép thêm và đáp án phụ thuộc vào trạng thái tại thời điểm hỏi.

Phần IV. Mười code mẫu đầy đủ

Mười file này là **đúng** nội dung trong solutions/ref/, tức đúng thứ bộ chấm chạy. Công tools/qa-note-code.py so từng dòng và trượt nếu hai bên lệch nhau. Chú thích viết bằng tiếng Anh vì bộ dụng bản in không đặt được tiếng Việt có dấu trong khối mã.

Bài A. Tổng và tích lớn nhất

```
# Problem A: the sum of the array, and the largest product of two distinct positions.
#
# In C++ this problem exists to punish 32 bit overflow twice over: the sum reaches
# 2*10^14 and the product reaches 10^18, so "long long p = a[i] * a[j];" is still wrong
# because the multiplication already happened in int.
#
# In Python that half of the problem is simply gone. Integers are arbitrary precision,
# so sum() and a*b are exact at any size. What is left is the half that was never about
# the type system at all: the best product may come from the two MOST NEGATIVE values,
# because a negative times a negative is positive. Sequence -10 -9 1 2 answers 90.
#
# The Python specific way to lose this problem is to let a value pass through float:
# sorted() on ints is exact, but max(a[-1] * a[-2], a[0] * a[1]) written with / or with
# float() anywhere silently rounds at about 9*10^15, which is INSIDE the 10^18 range of
# the answer. Integers never overflow here; floats still lose digits.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = sorted(map(int, data[1:1 + n]))
    tong = sum(a)
    # Only two candidates once the array is sorted.
    tich = a[n - 1] * a[n - 2]
    khac = a[0] * a[1]
    if khac > tich:
        tich = khac
    sys.stdout.write("%d %d\n" % (tong, tich))

main()
```

Bài B. Hạng của phần tử

```
# Problem B: the rank of every element among the distinct sorted values, 1 based.
# This is coordinate compression, the technique behind almost every Fenwick tree and
# segment tree over a large value range.
#
# The C++ recipe is four steps that must come in order: sort, unique, erase, lower_bound.
# Two of those four steps are places to lose the problem: unique() only removes ADJACENT
# duplicates so forgetting sort leaves most of them, and unique() does not shrink the
# vector so forgetting erase leaves garbage in the tail.
#
# Python replaces all four with sorted(set(a)), and both of those traps disappear with
# them. What replaces them is a cost question, not a correctness one. Two ways to map a
# value to its rank:
#     bisect_left(b, x) + 1           a binary search per element, O(n log n) total
```

```
#     hang[x]                                one dict built once, O(n) total
# The dict wins here because every element has to be looked up anyway. Building the dict
# with a comprehension keeps the loop at C speed.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = list(map(int, data[1:1 + n]))
    # sorted(set(a)) is the whole of sort + unique + erase, and it cannot be done in the
    # wrong order because there is only one call.
    hang = {v: i + 1 for i, v in enumerate(sorted(set(a)))}
    sys.stdout.write(" ".join([str(hang[x]) for x in a]) + "\n")

main()
```

Bài C. Đếm cặp có tổng cho trước

```
# Problem C: count index pairs i < j with a[i] + a[j] == X.
#
# One pass with a frequency table: standing at position j, every earlier element equal
# to X - a[j] makes a pair. Counting only what has already been seen gives i < j for
# free and handles duplicate values with no case analysis.
#
# What the C++ version spends its warnings on: the answer reaches 2*10^10 and X itself
# reaches 2*10^9, so both need long long, and unordered_map can be hacked into O(n) per
# operation. In Python all three of those are gone: integers are unbounded, and dict is
# the only hash table there is.
#
# What replaces them is one silent trap of dict itself. Writing seen[X - v] raises
# KeyError the first time the complement is missing, so the answer must be read with
# .get(key, 0), never with []. On a plain dict [] raises; on a defaultdict or Counter
# [] would instead CREATE the entry, which is the C++ std::map trap in a new coat and
# quietly doubles the memory of this solution.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    x = int(data[1])
    dap = 0
    da_thay = {}
    lay = da_thay.get
    for tok in data[2:2 + n]:
        v = int(tok)
        dap += lay(x - v, 0)
        da_thay[v] = lay(v, 0) + 1
    sys.stdout.write("%d\n" % dap)

main()
```

Bài D. Truy vấn đếm không vượt quá

```

# Problem D: for each query x, how many elements of the array are <= x.
# Sort once, then each query is one binary search: the count of elements <= x IS the
# index of the first element strictly greater than x, which is bisect_right.
#
# The C++ lesson here is the lower_bound / upper_bound pair. Python has the same pair
# under different names, and the same one place to lose the problem:
#     bisect_left(a, x) first index with a[i] >= x counts elements < x
#     bisect_right(a, x) first index with a[i] > x counts elements <= x
# The words "less than or equal" in the statement choose bisect_right. Test 01 has the
# query x = 1 against two ones in the array, so bisect_left answers 0 where the judge
# wants 2, on the sample.
#
# The Python specific note is about the loop, not the search. Both bisect functions are
# C code, so 2*10^5 calls cost almost nothing; but looking the function up through the
# module on every iteration does not, and neither does calling print() per query. Bind
# bisect_right to a LOCAL name before the loop and write the answers out once. Local
# names are looked up by index, module attributes by dictionary.
import sys
from bisect import bisect_right

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = sorted(map(int, data[1:1 + n]))
    p = 1 + n
    q = int(data[p]); p += 1
    tim = bisect_right # local name, looked up by index
    ra = [str(tim(a, int(t))) for t in data[p:p + q]]
    sys.stdout.write("\n".join(ra) + "\n")

main()

```

Bài E. Sắp xếp phân số

```

# Problem E: sort fractions a/b ascending, ties broken by the smaller denominator.
#
# The statement forbids floating point, and it means it: a double keeps 53 significant
# bits, about 15 to 16 decimal digits, while two distinct fractions with parts up to
# 10^9 can differ in the 18th digit. Test 04 is exactly that case, a pair of Farey
# neighbours built from Fibonacci numbers whose cross products differ by one.
#
# C++ answers with a comparator that cross multiplies: a/b < c/d is a*d < c*b when the
# denominators are positive. Python's sort takes a KEY, not a comparator, and that is
# the real difference between the two languages here. functools.cmp_to_key exists, but
# it wraps every element in an object and runs a Python level comparison for each of the
# roughly n*log2(n) comparisons, which is n calls of the key against about n*log2(n)
# calls of the comparator. Measured on 2*10^5 random fractions on this machine: integer
# key 0.210 s, cmp_to_key 0.443 s, fractions.Fraction as the key 1.690 s. All three are
# exact; only the first is the shape Python is built for.
#
# So the job is to turn "the exact value of a/b" into something sortable. Scale it into
# an integer: k = (a * 10^20) // b. Two DISTINCT fractions with parts up to 10^9 differ
# by at least 1/(b1*b2) >= 10^-18, which is 100 steps of the 10^-20 quantum, so the
# floors cannot collide; two EQUAL fractions give the same k because they are the same
# number. Python integers are unbounded, so the 10^20 scale factor costs nothing but a
# few extra limbs. This was checked against fractions.Fraction on 200000 random pairs
# plus adversarial Farey neighbours: zero disagreements.
import sys

```

```

THANG = 10 ** 20      # 100 times finer than the closest two legal fractions can be

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    f = []
    p = 1
    for _ in range(n):
        a = int(data[p]); b = int(data[p + 1]); p += 2
        # Key is (exact value scaled to an integer, denominator) so the tie rule of the
        # statement falls out of ordinary tuple comparison.
        f.append(((a * THANG) // b), b, a))
    f.sort()
    sys.stdout.write("".join(["%d %d\n" % (t[2], t[1]) for t in f]))

main()

```

Bài F. Gộp đồng chi phí nhỏ nhất

```

# Problem F: minimum total cost of merging all piles, which is Huffman merging.
# Greedy: always merge the two lightest piles, because a pile pays its own weight once
# for every merge it takes part in, so the heavy ones must join last.
#
# The C++ trap is syntax: priority_queue is a MAX heap and three template arguments are
# needed to flip it. Get it wrong and the program still runs and still prints a number,
# just the wrong one.
#
# Python has no such trap, because heapq is a MIN heap and there is no other kind. The
# mirror image bites instead: when a max heap is what you want, heapq cannot give you
# one and the standard trick is to push negated values.
#
# The Python cost note: heapify() turns the whole list into a heap in O(n) with a C
# loop, while n separate heappush calls are n Python level calls for the same result.
# And n = 1 must still print 0, which "while len(h) > 1" handles on its own; a loop
# written as "while h" would pop from an empty heap on the second pop.
import sys
from heapq import heapify, heappush, heappop

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    h = list(map(int, data[1:1 + n]))
    heapify(h)                                # O(n) in C, not n pushes from Python
    tong = 0
    while len(h) > 1:
        a = heappop(h)
        b = heappop(h)
        a += b
        tong += a
        heappush(h, a)
    sys.stdout.write("%d\n" % tong)

main()

```

Bài G. Bảng xếp hạng

```
# Problem G: leaderboard. Score descending, ties broken by name ascending, print top k.
#
# In C++ this is the two level comparator drill, and the drill exists because the shape
# is easy to get wrong: every level but the last must test for INEQUALITY first, and the
# last level must use a strict <. A comparator that answers true for two equal rows
# breaks the strict weak ordering contract and libstdc++ then reads past the end of the
# array, so the program crashes only on inputs big enough to matter.
#
# None of that exists in Python, and the reason is worth stating plainly: sort() takes a
# key, not a comparator, so there is no contract to break. A tuple key compares
# lexicographically for free, and sort() is stable, so the levels just stack up.
#
# Descending on one level and ascending on another is the one thing a key cannot express
# directly. For a number, negate it: -score. For a string you cannot negate, so the
# alternative is to sort by the ascending level FIRST and then re sort by the descending
# level, which stability makes correct. Here only the score needs flipping, so one sort
# with key (-score, name) does the job.
#
# Reading detail: the names stay as raw bytes from split(). Comparing bytes is the same
# lexicographic order as comparing the decoded ASCII strings, and it skips 2*10^5 decode
# calls, so the decoding happens only for the k rows that are actually printed.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    p = 2
    ds = []
    for _ in range(n):
        ten = data[p]; diem = int(data[p + 1]); p += 2
        ds.append((-diem, ten))
    ds.sort()
    ra = ["%s %d" % (t[1].decode(), -t[0]) for t in ds[:k]]
    sys.stdout.write("\n".join(ra) + "\n")

main()
```

Bài H. Vòng tròn

```
# Problem H: a cursor walking around a circle of n cells, q signed steps.
#
# This problem was built around two C++ traps, and BOTH of them are gone in Python.
#
# 1. Sign of the remainder. In C++ -1 % 5 is -1, so every answer has to be pushed back
#    into range with ((x % n) + n) % n. In Python % follows the DIVISOR, so -1 % 5 is
#    already 4 and the normalisation is dead code.
#
# 2. When to reduce. In C++ adding 2*10^5 steps of 10^18 before taking the remainder
#    overflows long long, so the reduction has to happen every step. Python integers
#    are unbounded, so a single accumulator and one % at the end is exact.
#
# So this is the one problem of the ten that is genuinely easier in Python, and it is
# worth understanding the rule underneath rather than just enjoying it. Python's // and
# % round toward NEGATIVE INFINITY; C++ truncates toward zero. -7 // 2 is -4 in Python
```

```

# and -3 in C++; -7 % 2 is 1 in Python and -1 in C++. Here that direction happens to be
# the one the problem wants. It is not always: a program that needs "how many whole
# groups fit" must not use // on a negative number, because -7 // 2 == -4 rounds AWAY
# from zero, and the fix there is -(-7 // 2) or int(-7 / 2) with the float caveat.
#
# Cost note: the running position is kept reduced anyway, not because it must be, but
# because letting an accumulator grow past 2*10^23 would drag every later addition into
# multi limb arithmetic for nothing.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])
    vt = 0
    ra = []
    them = ra.append
    for tok in data[2:2 + q]:
        vt = (vt + int(tok)) % n      # Python % is never negative when n > 0
        them(vt)
    sys.stdout.write("\n".join(map(str, ra)) + "\n")

main()

```

Bài I. Đếm từ trong câu

```

# Problem I: for each of n lines, print the word count and the first longest word.
#
# The C++ version of this problem is the cin.ignore() drill: after reading n with >>,
# the newline is still in the buffer and the first getline returns "". The Python shape
# of the same bug is real but different, and there are two ways into it.
#
# - readline() KEEPS the trailing newline, so a line looks one character longer than
#   it is, and printing it emits a blank line as well.
# - splitting the whole stream with .split() throws the line structure away entirely.
#   This problem is about LINES, so the stream is split on b"\n" and nothing else.
#
# Note what is NOT a problem here: leading, trailing and repeated spaces. In C++ that is
# what istringstream is for; in Python, split() with no argument already collapses every
# run of whitespace and drops the empty pieces at both ends, so " a  bb " gives exactly
# ["a", "bb"]. Calling .split(b" ") instead would produce empty strings and count them
# as words, which is the one way to get this wrong.
#
# The tie rule is "earliest wins", and max(..., key=len) returns the FIRST maximum it
# meets, so it matches for free; sorting by length would not guarantee it.
#
# A line with no words at all must print "0 -", and test 09 contains such lines. That
# case is also where a solution written as max(w, key=len) with no guard raises
# ValueError on an empty sequence and turns a WA into an RE.
import sys

def main():
    data = sys.stdin.buffer.read().split(b"\n")
    n = int(data[0])
    ra = []
    them = ra.append
    for i in range(1, n + 1):
        tu = data[i].split()

```



```

    if tu:
        them(b"%d %s" % (len(tu), max(tu, key=len)))
    else:
        them(b"0 -")
    sys.stdout.buffer.write(b"\n".join(ra) + b"\n")

main()

```

Bài J. Người bạn gần nhất

```

# Problem J: keep a set of values, answer "distance from x to the nearest stored value".
#
# The idea is the same in both languages: keep the values in sorted order, and the
# nearest value to x is one of the two neighbours of x in that order. Both neighbours
# can be missing, and the set can be empty, which is where most solutions die.
#
# C++ reaches for std::set, and the C++ lesson is about iterators: use the MEMBER
# s.lower_bound(x), because the free std::lower_bound walks a tree node by node and
# turns O(log n) into O(n); then check it != end() before dereferencing and it != begin()
# before stepping back.
#
# Python has no ordered set at all. That absence is the lesson. The stand in is a plain
# list kept sorted with the bisect module, and it trades a different cost: bisect_left
# is O(log n) like the tree, but inserting has to SHIFT the tail of the list, which is
# O(n) elements moved per insertion. It survives here because the shift is a single
# memmove down in C over a list small enough to stay in cache: test 09, with about 10^5
# insertions into a list growing to 10^5 entries, was measured at 0.707 s against the
# 3000 ms limit of this topic. Separately measured, the same 10^5 sorted insertions cost
# 0.625 s while sorting the same values in one call costs 0.038 s, so keeping a list
# sorted as you go is about sixteen times the price of sorting once. When the values are
# all known in advance, sort once instead. Here they are not: the queries interleave.
#
# Index arithmetic replaces the iterator ritual, and it has the same two boundary cases
# with clearer names: i == len(ds) means x is past the end, i == 0 means x is before the
# beginning, and an empty list makes both true at once, which is the -1 case.
import sys
from bisect import bisect_left

def main():
    data = sys.stdin.buffer.read().split()
    q = int(data[0])
    ds = []
    ra = []
    them = ra.append
    tim = bisect_left
    p = 1
    for _ in range(q):
        loai = data[p]; x = int(data[p + 1]); p += 2
        if loai == b"1":
            i = tim(ds, x)
            if i == len(ds) or ds[i] != x:    # insert only if not already present
                ds.insert(i, x)
            continue
        if not ds:
            them(-1)
            continue
        i = tim(ds, x)
        if i == len(ds):

```

```
        them(x - ds[i - 1])
    elif i == 0:
        them(ds[0] - x)
    else:
        a = ds[i] - x
        b = x - ds[i - 1]
        them(a if a < b else b)
    sys.stdout.write("\n".join(map(str, ra)) + "\n")

main()
```